Grafika komputerowa i komunikacja z komputerem

Laboratorium 4
3D transformacje geometryczne i wypełnianie obszarów.

ZADANIA DO WSTĘPNEGO PRZYGOTOWANIA (będą realizowane w trakcie zajęć)

Zadanie 1

Trójkąt o wierzchołkach

$$(0,0,0),\quad (1,0,0),\quad (0,1,0)$$

(tzn. pierwotnie znajdujący się w płaszczyźnie obrazu OXY) został przekształcony przez dwie różne transformacje 3D, tzn.:

1. Obrócony w przestrzeni 3D z użyciem kątów **RPY = [30º, 30º, 10º]** a następnie przesunięty o wektor **[0.5, 0.3, 1.9]**.
2. Obrócony w przestrzeni 3D z użyciem kątów **RPY = [-30º, -30º, -10º]** a następnie przesunięty o wektor **[0.3, 0.5, 2.0]**.

Jak wyglądają rzuty na płaszczyznę obrazu trójkąta w obu przypadkach, jeśli:

a. Stosujemy rzut równoległy wzdłuż osi OZ.

b. Stosujemy rzut perspektywiczny, gdzie oś OZ jest osią wirtualnej kamery, a ogniskowa tej kamery ma długość $f = 1$.

Wygeneruj odpowiednie obrazy cyfrowe, zakładając, że mają rozdzielczość **400x400** pixeli, a rozmiar pojedynczego pixela to **0.01x0.01**.

Wyświetl trójkąty w postaci jasnych linii (np. o jasności 200) łączących wierzchołki trójkąta na czarnym tle obrazu. Do kreślenia linii możesz użyć podanej w materiałach funkcji `linia1`. Jest to pełna i w miarę optymalna implementacja *algorytmu Bresenhama* (omawiany na Wykładzie 5).

Do realizacji tego zadania przydatne będą też dwie inne podstawowe funkcje zamieszczone w materiałach, tzn. `RPYT` oraz `Persp`.

Zadanie 2

Zrealizuj funkcję `floodfill(Im,y,x,T,C)` która w obrazie `Im` wypełnia jasnością `C` obszar o oryginalnej jasności `T`, w którym znajduje się pixel startowy o współrzędnych `(y,x)`. Jeśli pixel `(y,x)` ma inną jasność niż `T`, sygnalizuj błąd i przerwij działanie funkcji. Funkcja powinna zwracać obraz z poprawnie wypełnionym obszarem.

Sprawdź działanie funkcji na wybranych obszarach (i wybranych pixelach startowych) załączonego obrazu binarnego `BW.bmp`.

Zadanie 3

Zrealizuj Zadanie 1 w taki sposób, aby trójkąty wyświetlone były w postaci jasnych (np. o jasności 200) obszarów na czarnym tle obrazu. Wykorzystaj funkcję `floodfill` z Zadania 2.

Zadanie 4

Zrealizuj krótką animację, w której trójkąt z Zadania 1 będzie obracał się od orientacji RPY = **[0°, 0°, 0°]** do orientacji RPY = **[30°, 30°, 10°]** i z powrotem (można to zrobić kilka razy). Przyrost/zmniejszenie kątów obrotu w każdym kroku wynosi odpowiednio **[0.3°, 0.3°, 0.1°]**, tzn. animacja wymaga stu klatek na osiągnięcie docelowej orientacji, i stu klatek na powrót.

Wektor przesunięcia trójkąta jest cały czas taki sam, tzn. **[0.5, 0.3, 1.8]**.

Do otrzymania obrazu trójkąta zastosuj rzut perspektywiczny, gdzie oś OZ jest osią wirtualnej kamery, a ogniskowa tej kamery ma długość $f = 1$.

Klatki animacji mają rozdzielczość **400x400** pixeli, rozmiar pojedynczego pixela to **0.01x0.01**, a trójkąt ma być wypełniony (jak w Zadaniu 3). Dodatkowo załóż, że tło animacji nie jest czarne, tylko ciemnoszare (np. o jasności **50**).

Dla przypomnienia: Przykładowa sekwencja operacji używanych przy generowania pliku video wygląda następująco:

```
video = VideoWriter('zad4.avi');
open(video); % otwórz plik video
% POCZĄTEK PĘTLI
    % Wygeneruj obraz klatka i dopisz go do pliku wideo
    writeVideo(video,klatka);
% KONIEC PĘTLI
close(video);
```

Zadanie 5 (dla ambitnych)

Zrealizuj Zadanie 2 w postaci funkcji `floodfill_mod(Im,y,x,T,CIm)` która w obrazie `Im` wypełnia obszar o oryginalnej jasności `T`, w którym znajduje się pixel startowy o współrzędnych `(y,x)`. Wypełniamy dowolnym obrazem monochromatycznym `CIm`, przy czym pixel startowy malujemy na jasność `CIm(1,1)`, a pozostałe pixele malujemy w sposób zachowujący treść obrazu `CIm`. Jeżeli wykraczamy poza rozdzielczość obrazu `CIm`, należy użyć współrzędnych `modulo`.

Jeśli pixel `(y,x)` ma inną jasność niż `T`, sygnalizuj błąd i przerwij działanie funkcji.

Sprawdź działanie funkcji na wybranych obszarach (i wybranych pixelach startowych) załączonego obrazu binarnego `BW.bmp`, używając jako wypełnienia obrazu `okna.bmp`.